SpillFlow: Resource-Bounded External-Memory Sequence Operators for .NET

Abhishek Kore
Independent Researcher, Miami, Florida, USA
Email: abhikore84@gmail.com
August 24, 2026

Abstract

Large sequence operations often create sets, lookup tables, or sort buffers that grow with the input. This paper presents SpillFlow, an open-source .NET prototype that executes Distinct, ExceptBy, GroupJoin, grouped aggregation, and OrderBy with bounded library-owned memory and controlled temporary storage. The design combines comparer-compatible hash partitioning, recursive repartitioning, explicit memory leases, bounded multi-pass merge, pooled asynchronous block I/O, disk quotas, checksums, optional authenticated encryption, generated binary codecs, cleanup, and recovery metadata. In isolated 100-million-record tests under a 1 GiB managed-heap limit, SpillFlow was 1.51 to 4.66 times faster than ExternalSort 2.0.0-beta02 across four comparable operations, while producing matching output counts and checksums. A one-billion-record Distinct test completed in 18 minutes 19 seconds with a 154.8 MB peak working set; LINQ ran out of memory. These are single-run prototype results, not universal performance claims.

Keywords: external memory; out-of-core processing; .NET; bounded memory; asynchronous I/O; partitioning; merge sort

AI disclosure: Generative AI tools were used to assist with software implementation, test and benchmark development, literature discovery, language editing, and manuscript formatting. The author supplied the research goals and design direction, reviewed the outputs, verified the source code, tests, and reported results, and accepts responsibility for the paper.

1. Introduction

In-memory sequence APIs are easy to use and fast when working state fits in RAM. They become risky when the number of unique keys, join matches, or sortable records is not known in advance. A process can spend most of its time in garbage collection or fail before producing a result. External-memory algorithms solve this class of problem by moving bounded blocks between RAM and secondary storage [1, 2].

Mature external-memory systems exist, especially in C++. STXXL and TPIE show that blocked I/O, pipelining, and careful resource management can support data sets larger than RAM [3, 4]. SpillFlow does not claim to invent external merge sort or hash partitioning. Its contribution is a managed-runtime design that joins these ideas into familiar asynchronous .NET sequence operators and applies one resource policy across memory, temporary disk, integrity, and recovery.

This paper makes four contributions:

- A unified asynchronous API for five disk-backed sequence operations.

- A resource design that combines memory leases with recursive hash refinement and bounded merge fan-in.
- A versioned temporary-block layer with pooled I/O, quotas, CRC32, optional AES-GCM, generated codecs, cleanup, and recovery metadata.
- A reproducible prototype evaluation at 10 million, 100 million, and one billion records, with correctness checks and explicit limits.

2. Background and Related Work

The external-memory model counts transfers of blocks between a limited internal memory and a larger secondary store. Aggarwal and Vitter established fundamental I/O bounds for sorting and related problems [1], and Vitter later surveyed external-memory algorithms and programming environments [2]. External merge sort creates sorted runs in memory and then performs a k-way merge. Hash-based external operators send equal keys to the same partition so one partition can be handled at a time.

STXXL provides external-memory containers and algorithms for C++, overlaps I/O with computation, and supports pipelining and parallel disks [3]. TPIE provides a C++ framework for portable external-memory computation and later added a streaming pipeline model [4]. In the .NET ecosystem, the comparison package used here, ExternalSort 2.0.0-beta02, provides disk-backed operations but does not expose the same temporary-disk telemetry or grouped aggregation operation. SpillFlow focuses on a compact .NET API plus integrated operational controls.

.NET supplies useful building blocks rather than a complete external-sequence system. IAsyncEnumerable supports asynchronous iteration [5], ArrayPool permits reusable buffers [6], and the runtime can enforce a managed-heap hard limit [7]. SpillFlow composes these facilities with partition and merge algorithms.

3. Requirements and Threat Model

The design targets batch, analytics, claims, logging, and banking workloads with inputs that may exceed RAM. It requires:

- Correct results for custom equality comparers and key selectors.
- A bounded budget for retained memory owned by the library.
- A hard quota for physical temporary data and early deletion of consumed files.
- A bounded number of simultaneously open merge readers.
- Detection of accidental temporary-block corruption and optional confidentiality at rest.
- Cancellation, deterministic cleanup, and recoverable session metadata.

The memory budget is not a hard limit on total process RAM. The CLR, user callbacks, codecs, and operating system can allocate outside SpillFlow's leases. AES-GCM protects temporary block contents when enabled [8], but the calling application must protect the key. The prototype is not a key vault and does not promise secure deletion.

4. System Design

A SpillSession owns the resource policy. It tracks memory leases, disk reservations, temporary files, codecs, encryption material, statistics, cleanup, and recovery manifests. Every operation uses the same session, so a fast path and a spill path cannot silently apply different limits.

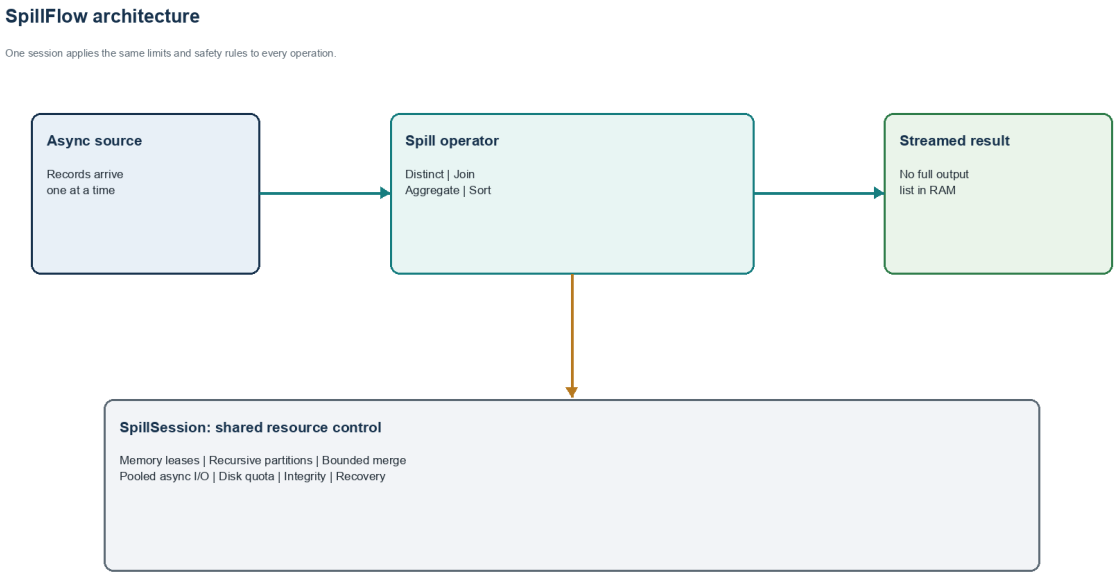


Figure 1. Main SpillFlow components and shared session controls.

Component	Responsibility
Memory governor	Reserves and releases bytes before large state or I/O buffers are used.
Partition layer	Routes equal keys together and recursively splits unsafe partitions.
Merge layer	Creates sorted runs and enforces a fixed merge fan-in.
Block I/O	Uses pooled buffers and framed asynchronous reads and writes.
Protection	Applies disk quota, CRC32, optional AES-GCM, manifests, and cleanup.

4.1 Memory leases and recursive refinement

A memory lease is permission to retain a known number of bytes. Before decoding a partition or renting a large buffer, the operation must obtain a lease. If the estimated decoded state will not fit, the partition refiner hashes the key at a deeper level and writes 16 child partitions by default. This repeats to a configured maximum depth. The operation either reaches a safe leaf or fails with a clear resource exception; it does not quietly load an oversized partition.

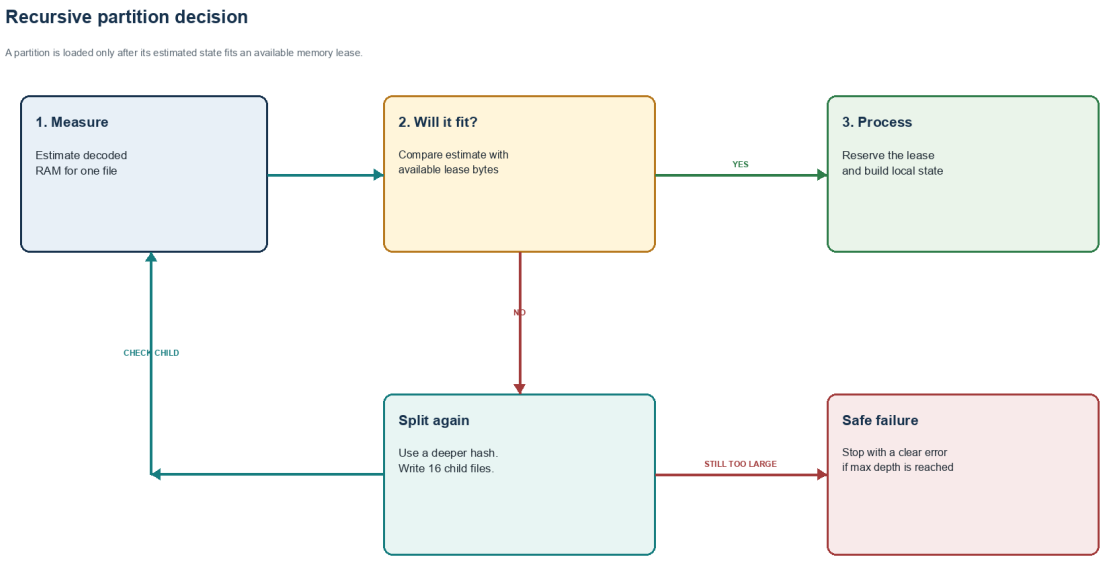


Figure 2. Recursive refinement under the memory-lease budget.

4.2 Block I/O and protection

Writing one record per system call is expensive. SpillFlow encodes records into an ArrayPool-backed block and writes a full block asynchronously. Each record has a length prefix. Each block has a versioned 48-byte header containing format identity, flags, lengths, CRC32, and encryption fields. The reader validates lengths before allocation, decrypts when required, verifies the checksum, and only then decodes records. Disk bytes are reserved before a physical write, so the session fails before crossing its quota.

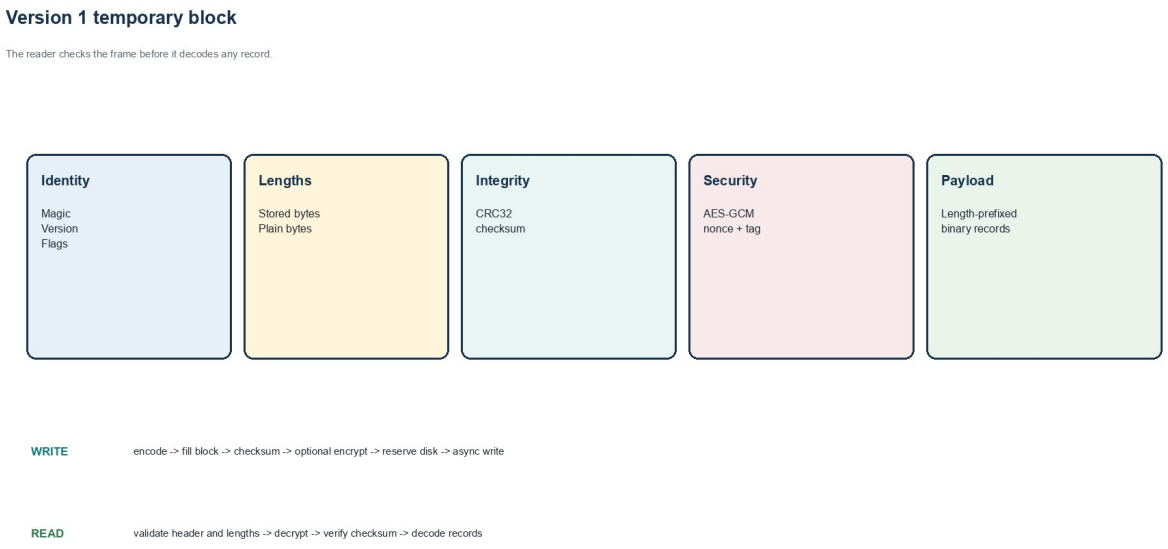


Figure 3. Version 1 temporary-block structure and read/write paths.

5. Operator Algorithms

Operator	External-memory plan	Bounded in-memory state
DistinctSpill	Stable hash partitions; recursive refinement	One set of seen keys for one leaf
ExceptBySpill	Paired source and exclusion partitions	Exclusion and seen sets for one leaf
GroupJoinSpill	Paired left and right partitions	Right-side lookup for one leaf
GroupAggregateSpill	RAM fast path, then hash partitions	One accumulator per key in one leaf
OrderBySpill	Sorted runs; bounded multi-pass merge	One run buffer and one item per open run

Hash operators obtain the partition hash from the supplied equality comparer. This is required because values considered equal must reach the same leaf. ExceptBy and GroupJoin refine paired files with the same deeper hash. Grouped aggregation begins in RAM and changes to spill mode only when its estimated state crosses the threshold. OrderBy fills a bounded list, sorts it, writes a run, and merges at most 16 runs at a time. More runs cause additional merge passes rather than an unbounded reader count. Final results are streamed.

6. Implementation

The prototype targets .NET 8 and exposes IAsyncEnumerable-based extension methods. Generated codecs use fixed little-endian numeric representations, UTF-8 strings with checked lengths, and deterministic property ordering. Unsupported or nonconstructible models produce a compile-time diagnostic. A JSON fallback remains available but is slower and less predictable.

Session disposal removes temporary files. Abandoned session directories can be removed after a configured age. A recovery manifest stores format versions, session identity, file names, disk accounting, and caller checkpoints. Checkpoint replacement uses a flushed temporary file followed by a move. Recovery validates versions and file names before accepting preserved data.

7. Experimental Method

Each benchmark case ran as a fresh Release process on 64-bit Windows with .NET SDK 8.0.421 and runtime 8.0.27. DOTNET_GCHeapHardLimit was set to 0x40000000, a 1 GiB managed-heap ceiling documented by Microsoft [7]. SpillFlow used a 64 MiB session memory budget. Inputs were generated lazily. The comparison used ExternalSort 2.0.0-beta02. Every completed pair had to return the same output count and deterministic checksum.

Reported wall time, total process CPU time, peak working set, managed allocations, SpillFlow peak leases, and SpillFlow peak temporary disk came from the benchmark harness. ExternalSort does not expose matching temporary-disk counters. The 100-million-record values and the one-billion-record values are single trials. Hardware model, cache state, background load, and storage details were not preserved in the benchmark record. Therefore the evaluation demonstrates feasibility and a measured comparison on one environment, not a statistically general performance ranking.

8. Results

Operation	Spill wall	External wall	Speedup	Spill peak RAM	External peak RAM
Distinct	56.4 s	262.7 s	4.66x	123.0 MB	143.6 MB
OrderBy	133.8 s	348.4 s	2.60x	70.1 MB	142.1 MB
ExceptBy	251.5 s	379.6 s	1.51x	124.8 MB	126.9 MB
GroupJoin	132.3 s	359.9 s	2.72x	153.0 MB	153.4 MB

Operation	Spill CPU	External CPU	Spill allocations	External allocations	Spill peak disk
Distinct	78.6 s	349.5 s	0.14 GB	23.59 GB	0.80 GB
OrderBy	183.1 s	477.7 s	0.08 GB	23.86 GB	0.96 GB
ExceptBy	337.3 s	524.6 s	19.43 GB	50.76 GB	1.65 GB
GroupJoin	184.0 s	484.9 s	15.35 GB	56.33 GB	1.76 GB

SpillFlow was faster in all four comparable 100-million-record cases. Distinct had the largest improvement. OrderBy used about half the peak working set. ExceptBy and GroupJoin had similar peak working sets across implementations, so their elapsed-time improvements came mainly from lower total CPU work and lower cumulative allocation. ExceptBy and GroupJoin still allocated 19.43 GB and 15.35 GB over the full run; bounded peak memory does not mean allocation-free execution.

8.1 Billion-record Distinct case

Implementation	Outcome	Wall time	CPU time	Peak working set	Managed allocations
SpillFlow	Completed	18m 19s	1,442.7 s	154.8 MB	0.91 GB
ExternalSort	Completed	60m 57s	4,968.8 s	176.4 MB	375.2 GB
LINQ	Out of memory	< 1m	n/a	about 645 MB	n/a

The input contained one billion integers and 100 million unique keys. SpillFlow and ExternalSort returned 100 million rows with checksum 4,999,999,950,000,000. SpillFlow was 3.33 times faster by wall time. It wrote 16.0 GB cumulatively, reached 8.13 GB peak temporary disk, and peaked at 15.8 MB of leased memory. LINQ failed under the same managed-heap ceiling.

Result verification: The benchmark harness sets unique keys to input count divided by ten, so this case creates exactly 100,000,000 distinct keys. The expected checksum is n(n-1)/2 = 4,999,999,950,000,000, which matches both completed implementations. The recorded wall-time ratio is 3,657 / 1,099 = 3.33. All table values match the retained benchmark summary. The raw JSON console records were not retained, so this review checks the recorded evidence and arithmetic rather than repeating the multi-hour run.

9. Correctness and Validation

The release solution built without compiler warnings. Twenty-five automated tests passed. Coverlet reported 88.7 percent line coverage and 81.8 percent branch coverage. Tests cover custom comparers, recursive partitions, merge fan-in, quotas, corruption detection, encrypted round trips, generated codecs, recovery, cancellation, and cleanup. A clean consumer project restored the packed NuGet artifact, generated a codec, round-tripped a record, and executed DistinctSpill. NuGet audit found no known vulnerable direct or transitive packages at the time of testing.

10. Limitations and Validity Risks

- The results are single runs on one incompletely recorded machine. Repeated trials, confidence intervals, cold and warm cache separation, and multiple SSD/HDD systems are still needed.
- MemoryLimitBytes controls SpillFlow-owned retained state, not every CLR, codec, callback, native, or operating-system allocation.
- A single extremely frequent key cannot be made smaller by hashing. GroupJoin needs a dedicated heavy-key streaming path.
- Recovery restores files and caller checkpoints but does not automatically resume every internal operator phase.
- Block headers do not yet carry codec and schema fingerprints.
- Temporary-directory permissions and secure deletion need platform-specific hardening for regulated deployments.
- Hash operators do not preserve source order. OrderBy is not stable for equal keys without a sequence tie-breaker.

- Only one .NET comparison package was evaluated; the paper does not establish superiority over databases or mature C++ systems.

11. Reproducibility

The source tree contains the benchmark harness, tests, package files, and exact result tables. A representative command pair is:

```
$env:DOTNET_GCHeapHardLimit='0x40000000'
dotnet run -c Release --project benchmarks/SpillFlow.ResourceBenchmarks -- spill distinct
100000000 64
dotnet run -c Release --project benchmarks/SpillFlow.ResourceBenchmarks -- external distinct
100000000 64
```

The public MIT-licensed source repository is available at https://github.com/abkore83/SpillFlow. The evaluated source is preserved at commit cc2007c4c99f8a8374a4686bd8d21a1536a8f4e2. The current package version is 0.1.0-alpha.1. Future publication-quality runs should also preserve machine specifications and raw per-trial result files.

12. Conclusion

SpillFlow shows that familiar .NET sequence operations can process inputs far larger than RAM when every temporary resource is governed. Its main value is the integration of external-memory algorithms with managed-runtime controls: comparer-compatible partitioning, recursive refinement, explicit leases, bounded merging, pooled asynchronous blocks, quotas, integrity checks, optional encryption, generated codecs, cleanup, and recovery metadata. The prototype completed all measured large cases and substantially reduced elapsed time and cumulative allocation relative to the tested ExternalSort version. The evidence is promising but preliminary. The next publication-quality evaluation should preserve full hardware metadata, run repeated trials on several systems, test skewed data, and compare against additional engines.

Statements and Availability

Funding: This work received no federal or other sponsored research funding.

Competing interests: The author declares no competing financial or non-financial interests.

Code and data availability: Source code, benchmark harnesses, automated tests, package artifacts, and detailed benchmark results are publicly available under the MIT license at https://github.com/abkore83/SpillFlow. The version evaluated in this paper is commit cc2007c4c99f8a8374a4686bd8d21a1536a8f4e2.

References

[1] A. Aggarwal and J. S. Vitter, “The Input/Output Complexity of Sorting and Related Problems,” Communications of the ACM, vol. 31, no. 9, pp. 1116–1127, 1988. doi: 10.1145/48529.48535.

[2] J. S. Vitter, “External Memory Algorithms and Data Structures: Dealing with Massive Data,” ACM Computing Surveys, vol. 33, no. 2, pp. 209–271, 2001. doi: 10.1145/371578.371580.

[3] R. Dementiev, L. Kettner, and P. Sanders, “STXXL: Standard Template Library for XXL Data Sets,” Software: Practice and Experience, vol. 38, pp. 589–637, 2008. doi: 10.1002/spe.844.

[4] L. Arge et al., “External Memory Pipelining Made Easy With TPIE,” arXiv:1710.10091, 2017. https://arxiv.org/abs/1710.10091.

[5] Microsoft, “IAsyncEnumerable<T> Interface,” Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.collections.generic.iasyncenumerable-1.

[6] Microsoft, “ArrayPool<T> Class,” Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.buffers.arraypool-1.

[7] Microsoft, “Runtime Configuration Options for Garbage Collection,” Microsoft Learn. https://learn.microsoft.com/dotnet/core/runtime-config/garbage-collector.

[8] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, 2007. doi: 10.6028/NIST.SP.800-38D.